OPENPI · FLASHRT · 螺丝分拣 · 2026-11-06

同一份仓库，x86 训练，板上加速。

x86 和 Orin 跑的都是这份 OpenPI。权重是 `pi05_cobot_screw_sorting_single` 的 29999。机器人仍连 8000，客户端不改。FlashRT 只替换板上这一个进程。

流程 · 关键技术 · 1 转换 · 2 统计量 · 3 精度 · 4 换 PyTorch · 5 板上编译 · 6 FlashRT 服务 · 7 开关

方法流程

上面四步在 x86 完成。下面两步只在要加速的那块板上做。8000 上始终只有一个策略进程。



客户端 main.py 不动。只有把进程从一台机器挪到另一台时，才改 host。

关键技术

动作

先反归一化，再把增量加回关节

模型吐出的是分位数空间里的数。训练时前 6 维先减去当前关节，变成增量，夹爪保持绝对值，然后才做 q01/q99。反归一化只把这个增量还原成弧度，还不是电机目标。



环境

两套 Python，不要装进对方

x86 的 `uv` 里是 JAX 和 `torch==2.7.1`。板上的 `~/flashrt-venv` 里是 Jetson 轮子和 `flash_rt`。在 FlashRT 环境里 `pip install -e` 这份仓库，会把板上的 PyTorch 盖掉。

FlashRT 服务不再 import JAX。那个 venv 只再补 `websockets msgpack osqp scipy`。用它的 python 跑脚本，不要 `uv run`。

形状

50 是动作块，10 是去噪步

这份配置 `action_horizon=50`、`action_dim=32`。前 7 维才是右臂。官方 FlashRT 示例的 `(10, 7)` 是 LIBERO。时间维如果是 10，说明加载错了头，不要接电机。

相机是三路槽位：头顶、**全黑的左腕**、右腕。`num_views` 保持 3。收成 2 会让 token 错位。

不必改脚本的三处

和 serve_policy 的已知差别

左腕在正式路径里 `image_mask=False`。FlashRT 的 `predict` 没有 mask，黑图仍会被看见。接口补不上，不要为此改成两路相机。

缩放到 224 已在 `env.py` 里做完，正式服务再缩一次没有变化。`Task: …, State: …` 交给 FlashRT 的 `state=`，脚本里再包一次会把状态离散两遍。

这份权重

配置	JAX	提示词	数据集
pi05_cobot_screw_sorting_si…	checkpoints/pi05_cobot/.../29999	句号要留着，与现有启动脚本一致	130 集，53231 帧，30 Hz，480×640
现有入口	不碰		
run_server_screw_sorting_single.sh	9101 的 RLT actor，也不写回 29999		

1. 在 x86 上转成 safetensors

1 X86 · /HOME/XLK/WORK/OPENPI · UV 环境

只读 29999，不改它。不要在板上跑，板上没有这套 JAX。

```
cd /home/xlk/work/openpi
uv run examples/convert_jax_model_to_pytorch.py \
  --checkpoint_dir /home/xlk/work/openpi/checkpoints/pi05_cobot/pi05_cobot_screw_sorting_single/pi05_cob
  --config_name pi05_cobot_screw_sorting_single \
  --output_path /home/xlk/work/openpi/checkpoints/pi05_cobot_screw_sorting_single_pytorch
```

`--checkpoint_dir` 指到含有 `params/` 的 29999，不要指进 `params`。

做完应看到

`config.json` 里 `action_dim` 是 32，`action_horizon` 是 50，`precision` 是 `bfloat16`，旁边有 `model.safetensors`。

2. 补上统计量和分词器

2

转换脚本到上一级目录找 `assets/`，会静默跳过。29999 的统计量在自己目录里，必须手动拷。π0.5 用 q01/q99，不用 mean/std。

```
OUT=/home/xlk/work/openpi/checkpoints/pi05_cobot_screw_sorting_single_pytorch
mkdir -p "$OUT/assets/screw_sorting_single"
cp /home/xlk/work/openpi/checkpoints/pi05_cobot/pi05_cobot_screw_sorting_single/pi05_cobot_screw_sorting
   "$OUT/assets/screw_sorting_single/norm_stats.json"
ls /home/xlk/work/openpi/models/paligemma_tokenizer/paligemma_tokenizer.model
```

做完应看到

状态和动作的 `q01` 长度都是 7。分词器用训练时这份 SentencePiece，拷到板子的 `/data/checkpoints/paligemma_tokenizer.model`，并设 `FLASH_RT_PALIGEMMA_TOKENIZER`。

3. 在真实帧上对 JAX

3 X86 · 数据集 /DATA/HUGGINGFACE/LEROBOT/OPENPI/SCREW_SORTING_SINGLE_SFT

均匀抽 8 帧。两边吃同一张图、同一个 7 维状态、同一段噪声，再走 `serve_policy` 的同一套输出，包括 QP。比的是机械臂会收到的 7 维指令。示教余弦只作参照，两边接近即可，不必等于 1。

```
cd /home/xlk/work/openpi
uv run scripts/eval_screw_sorting_conversion.py \
  --pytorch_dir /home/xlk/work/openpi/checkpoints/pi05_cobot_screw_sorting_single_pytorch \
```

```
  --num 8 \
  --out /tmp/screw_sorting_conversion.json
```

通过线

`cosine_min ≥ 0.99`。低于这条脚本会退出，不要换服务器。打印的毫秒是这台 x86 的耗时，不是板上的指标。第一次调用是编译，不计入。

4. 在当前部署机上换成 PyTorch

4

x86 和 Orin 现在都是 `run_server_screw_sorting_single.sh`，读 JAX。余弦过线之后，在正在对外服务的那台机器上停掉它，改跑：

```
bash examples/mobile_aloha_AgileX/run_server_screw_sorting_single_pytorch.sh
```

还是 `serve_policy.py`，还是 8000，还是原来的观测和 QP，只是目录里换成 `model.safetensors`。这是 OpenPI 自己的 PyTorch，不是 FlashRT。Orin 上先把第 2 步的目录拷过来。客户端 host 不用改。

拷到另一台机器时

`rsync -a` 整个 pytorch 目录，以及分词器文件。不要把这个目录指回 29999，也不要把板子上可能多出来的缓存同步回去覆盖 JAX。

5. 只在板上编译 FlashRT

5 ORIN 或 THOR · ~/FLASHRT-VENV · 不要用 UV

```
python3 -m venv ~/flashrt-venv
source ~/flashrt-venv/bin/activate
python -m pip install -U pip
# 先装本机 JetPack 对应的 aarch64 PyTorch。Orin 不要装 x86 的 cu124 轮子。
pip install pybind11 cmake "numpy>=1.24" safetensors "transformers<4.56" pandas pillow pyarrow
git clone https://github.com/flashrt-project/FlashRT.git
cd FlashRT
git clone --depth 1 --branch v4.4.2 https://github.com/NVIDIA/cutlass.git third_party/cutlass
pip install -e ".[torch]"
```

`transformers` 从 4.56 起分词路径坏了。可编辑安装必须在 `cmake` 之前，这样 `.so` 才会落进这个环境。

Orin · SM87

没有 FP8。快路径是 INT8。用 `-j4`，避免统一内存被编译打满。

```
sudo nvpmodel -m 0 && sudo jetson_clocks
export PATH=/usr/local/cuda/bin:$PATH
export CUDACXX=/usr/local/cuda/bin/nvcc
cmake -B build_orin_sm87 -S . -DGPU_ARCH=87 \
  -DFA2_ARCH_NATIVE_ONLY=ON \
  -DFA2_HDIMS='96;128;256' -DFA2_DTYPES='bf16' \
  -DPython3_EXECUTABLE="$(which python)"
cmake --build build_orin_sm87 -j4
```

Thor · SM110

默认 FP8。不生成 `flash_rt_fa2.so` 是预期。x86 的 flashrt 镜像不能拿来跑。

```
sudo jetson_clocks
cmake -B build -S . -DGPU_ARCH=110
cmake --build build -j$(nproc)
```

做完应看到

`python -c "import torch; print(torch.cuda.is_available())"` 为 True，并且能 `import flash_rt`。同一块板、同一个 JetPack，已经能 import 就不要重编。

6. 用 FlashRT 接上原来的 8000

6

先停掉 `serve_policy`。仍在 `~/flashrt-venv` 里，不要 `uv run`，也不要 `pip install -e` 这份 OpenPI。

```
source ~/flashrt-venv/bin/activate
pip install websockets msgpack osqp scipy
export FLASH_RT_PALIGEMMA_TOKENIZER=/data/checkpoints/paligemma_tokenizer.model
cd /home/xlk/work/openpi
python scripts/serve_flashrt_screw_policy.py \
  --checkpoint /data/checkpoints/pi05_cobot_screw_sorting_single_pytorch \
  --port 8000
```

脚本自己做这几件事，和 `policy_config.py` 的输出对齐：7 维状态按 q01/q99 归一化后连同裸提示词交给 FlashRT；回来后反归一化；前 6 维加上当前关节；夹爪不加；QP 权重 `w_data=1`、`w_acc=100`、`w_jerk=100`。Orin（计算能力 8.7）会在进程内打开 `FVK_PI05_RTX_FORCE_INT8`，退出即失效。

提示词与现有服务逐字相同，包括句号：`Please sort and return the silver screws in the grey box to their proper places.`

三件不要在脚本里再补

不要自己再包一层 `Task: …, State: …`。不要把 `num_views` 改成 2 来假装左腕 mask。480×640 的原始图不要直接送进这个进程，客户端已经在 `env.py` 缩到 224。RTC 会直接拒绝，当前这条螺丝服务本来也没开。

上电前

时间维必须是 50。前 6 维是绝对关节角，夹爪落在统计量里大约 0 到 0.036。空跑确认前 32 步不越限位，再低速跟一块。客户端每次仍只执行 32 步，不要在板上先砍成 10。

7. 板上开关，第一版就用这一列

开关	这份模型	原因
`num_views`	3	头顶、黑左腕、右腕。2 会错位
`num_steps`	10	去噪步数，与 `sample_actions` 默认一致
`vision_num_layers` / `pool`	27 / 1	少一层或池化会先抹掉小螺丝
`cache_frames`	Orin 第一版 1	1 与 BF16 逐 bit 相同。2 复用上一帧视觉，工件在动时先不要开
`FORCE_INT8`	只在 Orin	SM87 没有 FP8。脚本已按计算能力设置
`use_fp8`	Thor 第一版	NVFP4 等这一版的形状和空跑都对了再开，且必须带 AWQ
`state_prompt`	fixed / 200	对齐 `max_token_len`。状态一变，exact 会重捕图

Orin 上别人公布的数字

DROID、2 相机、`cache_frames=1`：124 ms，8.04 Hz，与 BF16 逐 bit 相同。这份权重是 3 路

Thor 上别人公布的数字

2 相机 FP8：纯 graph 40.71 ms，quickstart 约 44 ms。3 相机 encoder NVFP4 是 51.51 ms。协

槽位、动作块 50，会更慢。用自己的 p50，不要拿 124 ms 当验收线。

议不同就不要相减。锁频后再计，预热至少 20 次。

污染边界

脚本放在仓库里不会改训练，也不会被 `serve_policy` 引用。不要把 FlashRT 装进 uv。不要把 `--checkpoint` 指到 29999。8000 上先停再起，端口被占会直接失败，不会悄悄替换原来的服务。

权重与提示词来自 `run_server_screw_sorting_single.sh` 和 `pi05_cobot_screw_sorting_single`。增量动作来自 `use_delta_joint_actions` 与 `AbsoluteActions`。输出顺序对照 `policy_config.py`。板上延迟来自 FlashRT 的 Orin 指南和 README，测的是别的 checkpoint。